BO

- Introduction to BO
- Surrogates in BO

BO problems

An optimization problem is denoted with

$$\min_{x \in \mathcal{X}} f(x) \qquad \text{or} \qquad \max_{x \in \mathcal{X}} f(x)$$

The objective is a function $f : \mathcal{X} \mapsto \mathbb{R}$ *...this is computable but very expensive*

The search space is a set $\mathcal{X} \subset \mathbb{R}^d$ *...this must be bounded*

The queries are points $x \in \mathcal{X}$

In Bayesian optimization, the objective is relatively expensive to compute

...a critical requirement is the evaluation of the objective only a few times

In Bayesian optimization, the objective is replaced by a probabilistic surrogate

...a surrogate is an inexpensive approximation of the objective

...a probabilistic surrogate provides a full probability distribution for every query

In Bayesian optimization, the surrogate is used to guide the selection of query points

...new queries provide data used to refine the surrogate

...the refined surrogate is used to select new queries

In Bayesian optimization, the policy is the rule that leads from existing data to new queries

...this includes the refinement of the surrogate and its usage

BO surrogates

We consider an optimization problem

$$\min_{x \in \mathcal{X}} f(x) \qquad \text{or} \qquad \max_{x \in \mathcal{X}} f(x)$$

for an objective f on a search space $\mathcal{X}$ *...our objective must be real and scalar*

- Our optimization policy is based on a surrogate *...we seek to establish $g \approx f$*

$$g \sim \mathrm{GP}_{\mathcal{X}}(m, C)$$

for an appropriate mean $m : \mathcal{X} \mapsto \mathbb{R}$ and kernel $C : \mathcal{X} \times \mathcal{X} \mapsto \mathbb{R}$

- Our data are based on a likelihood *...our likelihood allows for $N \to \infty$ and $\sigma \to 0$*

$$y_n | g \sim \text{Normal}\left(g(x_n), \sigma^2\right), \qquad n = 1, \ldots, N$$

for an appropriate noise $\sigma \in \mathbb{R}$

- Our framework is determined by pre-selected m, C and σ

BO loops

We consider an optimization problem

$$\min_{x \in \mathcal{X}} f(x) \qquad \text{or} \qquad \max_{x \in \mathcal{X}} f(x)$$

with a surrogate

$$g \sim \mathrm{GP}_{\mathcal{X}}(m, C)$$
$$y_n | g \sim \mathrm{Normal}\left(g(x_n), \sigma^2\right), \qquad n = 1, \ldots, N$$

BO starts with:

- a pre-selected set of queries $\{x_n\}_{n=1}^{N^0} \subset \mathcal{X}$ — *...initial queries are obtained by space-filling designs*
- a pre-evaluated set of targets $\{y_n\}_{n=1}^{N^0} \subset \mathbb{R}$ — *...targets are obtained by the objective, $y_n = f(x_n)$*

BO proceeds iteratively: *...at iteration $j = 1, 2, \ldots$*

- form posterior surrogate using existing data — $g(x) | y_{1:N^{j-1}} \sim \mathit{Normal}\left(m(x; y_{1:N^{j-1}}), v(x; y_{1:N^{j-1}})\right)$
- apply the surrogate to select new query — *...our new query is x_n with $n = N^0 + j$*
- evaluate new target — *...our new target is obtained by the objective, $y_n = f(x_n)$*
- update data — *...our new data consist of $\{(x_n, y_n)\}_{n=1}^{Nj} \subset \mathcal{X} \times \mathbb{R}$ of size $N^j = N^0 + j$*

Space filling designs

We consider a bounded search space $\mathcal{X} \subset \mathbb{R}^d$ and seek a space-filling design

...space-filling designs spread queries near-evenly without large gaps or clusters

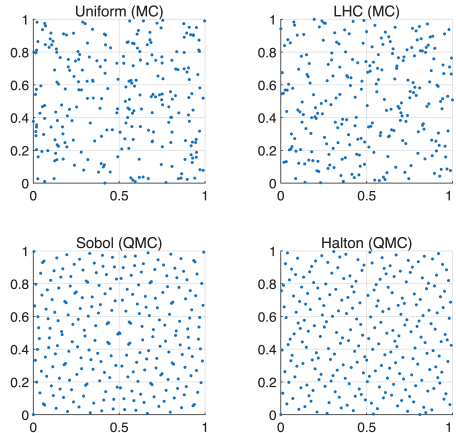
Monte Carlo space-filling designs include

- Uniform sampling *...highly inefficient*
- Latin hyper-cube sampling *...improved coverage*

Quasi-Monte Carlo space-filling designs include

- Sobol sets *...preferable in high dimension*
- Halton sets *...preferable in low dimension*

Sobol and Halton sets are low-discrepancy quasi-random sequences



Query selection

Within our BO loop, at iteration $j = 1, 2, \ldots$ we have data and posterior surrogate

$$\{(x_n, y_n)\}_{n=1}^{N^{j-1}} \subset \mathcal{X} \times \mathbb{R}$$
$$g(x)|y_{1:N^{j-1}} \sim \text{Normal}\left(m(x; y_{1:N^{j-1}}), v(x; y_{1:N^{j-1}})\right)$$

To balance exploitation/exploration across the search space, we derive an acquisition function $\alpha : \mathcal{X} \mapsto \mathbb{R}$

- For maximization problems

$$\max_{x \in \mathcal{X}} f(x)$$

the upper confidence bound is

$$\alpha_{UCB}(x; y_{1:N^{j-1}}) = m(x; y_{1:N^{j-1}}) + \beta\sqrt{v(x; y_{1:N^{j-1}})}$$

and the next query is selected by

$$x_{N^j} = \text{argmax}_{x \in \mathcal{X}} \alpha_{UCB}(x; y_{1:N^{j-1}})$$

- For minimization problems

$$\min_{x \in \mathcal{X}} f(x)$$

the lower confidence bound is

$$\alpha_{LCB}(x; y_{1:N^{j-1}}) = m(x; y_{1:N^{j-1}}) - \beta\sqrt{v(x; y_{1:N^{j-1}})}$$

and the next query is selected by

$$x_{N^j} = \text{argmin}_{x \in \mathcal{X}} \alpha_{LCB}(x; y_{1:N^{j-1}})$$

In both cases, $\beta > 0$ controls the exploitation/exploration trade-off

Demonstration

